COLLEGE OF SAINT AMATIEL INC.
DIPLOMA IN INFORMATION TECHNOLOGY
118 Int. Gen. Luna St., San Agustin, Malabon City
Email: saintamatieledu@gmail.com

SIS

STUDENT INFORMATION SYSTEM

SYSTEM DOCUMENTATION

Subject: System Integration and Architecture

SUBMITTED BY

Lucmayan, Joshan A.

Andres, Prince Kenneth A.

Tenorio, Nadine

Arellano, Ericka

Salvoro, Miguel

Rivera, John Carlo B.

Salamante, Chris Jordan

Instructor: Mr. Rogelio Galangue

Academic Year 2026-2027
October 2026

COLLEGE OF SAINT AMATIEL INC.
DIPLOMA IN INFORMATION TECHNOLOGY
118 Int. Gen. Luna St., San Agustin, Malabon City
Email: saintamatieledu@gmail.com

Contents

1. Introduction 3
1.1 Purpose . . . 3
1.2 Scope . . . 3
1.3 Intended audience . . . 3
1.4 Definitions . . . 3
2. System Overview 3
2.1 Goals . . . 3
2.2 Modules . . . 4
2.3 Roles . . . 4
2.4 Technology stack . . . 4
3. System Architecture 4
3.1 Runtime topology . . . 4
3.2 Request lifecycle . . . 5
3.3 Project structure . . . 5
3.4 Client architecture . . . 5
3.5 Server architecture . . . 5
3.6 Transactions and numbering . . . 6
4. Data Model 6
4.1 Tables . . . 6
4.2 Relationships . . . 6
4.3 Derived values . . . 6
4.4 Data lifecycle . . . 7
5. Functional Modules 7
5.1 Authentication and session . . . 7
5.2 Dashboard . . . 7
5.3 Student records . . . 7
5.4 Enrollment . . . 7
5.5 Courses and grades . . . 7
5.6 Attendance . . . 7
5.7 Library . . . 8
5.8 Billing . . . 8
5.9 Notifications . . . 8
5.10 Reports and audit . . . 8
5.11 Settings and appearance . . . 8
5.12 Integration flows . . . 8
6. API Reference 8
6.1 Status codes . . . 9
6.2 Example requests . . . 9
7. Security and Access Control 10
7.1 Authentication . . . 10
7.2 Authorization . . . 10
7.3 Input validation . . . 10
7.4 Audit and transport . . . 10
7.5 Known security limits . . . 10
8. Installation and Configuration 11
8.1 Prerequisites . . . 11
8.2 Install and run . . . 11
8.3 Configuration . . . 11
8.4 Seed accounts . . . 11
9. Testing and Quality Assurance 11
9.1 Test suite . . . 11
9.2 Results . . . 12
9.3 Manual acceptance walkthrough . . . 12
10. Limitations and Future Work 12
10.1 Current limitations . . . 12
10.2 Suggested next steps . . . 12
11. Appendix: Document Control 13

1. Introduction

1.1 Purpose

This document describes the design, implementation, and operation of the Student Information System (SIS). It explains how the browser application, the REST API, and the relational database fit together, what each module does, how the modules share data, how access is controlled, and how the system is installed, configured, and tested.

1.2 Scope

SIS serves a single campus with three kinds of users: administrators (registrar and bursar functions), librarians, and students. It covers student records, course catalog and enrollment, attendance, grades, library circulation, invoicing and payments, notifications, reports, and an audit trail.

The following items are outside the current scope: real payment processing, real email or SMS delivery, multi-campus tenancy, document uploads, and production hardening such as login rate limiting and refresh-token rotation. These limits are restated in Section 10.

1.3 Intended audience

The document is written for instructors and reviewers who need to understand the system, for future maintainers who will extend it, and for technical staff who will run it. Readers are expected to be familiar with web applications and relational databases.

1.4 Definitions

Term	Meaning
SIS	Student Information System, the subject of this document.
SPA	Single-page application. The React client is loaded once and routed in the browser.
API	Application programming interface. Here, the Express REST endpoints under /api.
JWT	JSON Web Token. A signed token that carries the user id, role, name, and email.
RBAC	Role-based access control. Permissions are attached to admin, librarian, or student.
LMS	Learning management features in this system: course catalog, rosters, and grades.
Seed data	The demonstration dataset created the first time the database is initialized.
CRUD	Create, read, update, delete.

2. System Overview

2.1 Goals

The system is built around one idea: every campus office should work from the same record. The registrar creates a student once; the bursar sees the same account and the same invoice; the library sees the same borrower; the student sees the same information in a personal portal. The design therefore favors strong relational constraints and transactional writes over loose, duplicated copies of data.

- One student record, one login, and one balance across every module.
- Automatic side effects: creating a student provisions a login, enrollments, and an invoice.
- Shared data is computed from the database rather than copied into reports.

- Every write is checked on the server, not only hidden in the interface.
- The whole system starts with one command and no external database installation.

2.2 Modules

Module	What it does	Primary users
Dashboard	Role-specific summary of campus or personal activity.	All
Students	Search, create, edit, and remove student records and portal logins.	Admin
Enrollment	Manage the student-course join records and their status.	Admin
Courses (LMS)	Course catalog, class rosters, enrollment, and grade entry.	Admin, students
Attendance	Daily roll call with present, late, absent, and excused states.	Admin, students
Library	Book catalog, loans, returns, availability, and overdue detection.	Admin, librarian, students
Billing	Invoices, partial and full payments, balances, and overdue status.	Admin, students
Notifications	Personal inbox, unread counts, and announcements.	All
Reports	Institution-wide academic, financial, and library reporting.	Admin
Settings	Light and dark appearance for the portal.	All
Integration / About	Architecture and API documentation inside the product.	All

2.3 Roles

Administrators manage records across the campus. Librarians manage the catalog and circulation only. Students see their own profile, courses, attendance, grades, library loans, invoices, and notifications. The sidebar, the route guards, and every API route enforce the same boundaries.

2.4 Technology stack

Layer	Technology	Responsibility
Client	React 19, React Router 7, Vite 7	Role-aware single-page interface and client routing.
Styling	Hand-written CSS with custom properties	Design tokens, light and dark themes, component styles.
API	Node.js, Express 5	REST endpoints, validation, authorization, audit.
Database	MySQL 8.4, InnoDB, utf8mb4	Relational storage, foreign keys, transactions.
Driver	mysql2/promise	Connection pool and transactional connections.
Local database	mysql-memory-server	Downloads and runs MySQL without a system install.
Authentication	JWT HS256, bcryptjs	Signed sessions and password hashing.
Tests	node:test and fetch	Unit and API integration tests with an isolated database.

3. System Architecture

3.1 Runtime topology

In development, two processes run: Vite serves the client on port 5173 and proxies /api requests to Express on port 4000. In a production-style run, Express serves the built client from client/dist and answers API requests on the same origin, so the browser needs only one server.

```
Browser (React SPA)
  |  fetch /api/* with Bearer token
  v
Express API (:4000)
  |  auth middleware -> role guard -> route handler
  |  mysql2 pool (10 connections)
  v
MySQL 8.4 (embedded by default, or an external server)
```

3.2 Request lifecycle

- The JSON body is parsed; malformed JSON becomes a 400 response.
- The audit middleware registers a finish listener for non-GET API calls.
- The auth middleware verifies the bearer token and sets req.user.
- The router applies role checks (adminOnly, allowRoles, or ownership checks).
- The handler runs queries or a transaction through the pool.
- The response is sent, then the audit row is written without blocking the response.
- Errors flow to one handler; HttpError keeps its status, anything else becomes a 500.

3.3 Project structure

```
sis/
  client/                 React single-page application
    src/components/       layout, UI primitives, icons, logo, charts
    src/context/          auth, toast, and theme providers
    src/lib/              API client, formatters, constants
    src/pages/            one screen per route
    src/styles.css        tokens, light and dark themes, components
  server/                 Express API
    src/middleware/       authentication, authorization, audit
    src/routes/           one router per module
    src/sql/schema.sql    tables, keys, constraints
    src/utils/            errors, validation, dates, notifications, sequences
  tests/                  node:test helper and API suites
  docs/                   architecture, API reference, this PDF
```

3.4 Client architecture

The client is a React SPA. main.jsx mounts the router and three providers: Theme, Auth, and Toast. App.jsx declares the routes and wraps restricted pages in RequireRole or AdminOnly, which redirect to the home route when the signed-in role does not match. Layout renders the role-specific sidebar, a topbar with the unread count, and the page outlet. Pages call the shared api helper, which attaches the token, parses JSON, and broadcasts a session-expired event on a 401 response.

The appearance preference is stored in localStorage under sis-theme. A small inline script in index.html applies the saved value before React renders, preventing a flash of the wrong theme. Settings exposes the same choice through ThemeContext.

3.5 Server architecture

Each module is a standalone Express router mounted under /api. Shared behavior lives in middleware and utilities: auth verifies tokens and exposes role guards, audit records write attempts, http defines the error type and validation helpers, dates normalizes local calendar dates, notify inserts notification rows, and sequence hands out student and invoice numbers atomically. Database access goes through a single pool; operations that must be atomic borrow a connection and run a transaction.

3.6 Transactions and numbering

Creating a student, creating an invoice, deleting an enrollment, saving an attendance sheet, and recording a loan or return all use one connection and one transaction. Student and invoice numbers come from a counters table using an atomic increment and LAST_INSERT_ID, so two concurrent requests cannot receive the same number. The first allocation seeds the counter from the highest existing number, which keeps imported records from colliding.

4. Data Model

The database contains 13 InnoDB tables: 12 domain tables plus a counters table for number allocation. Foreign keys connect accounts to students, students to their academic and financial records, and invoices to payments. Deleting a user cascades through the student record and its dependent rows.

4.1 Tables

Table	Purpose	Key constraints
users	One login per person, with role and active flag.	email unique; role enum
students	Academic profile linked to a user.	user_id unique; student_no unique
courses	Course catalog shared by programs.	code unique
enrollments	Student-course link with status.	unique student and course pair
attendance	One roll-call record per student, course, and day.	unique triple; date index
grades	Individual assessment scores.	student and course foreign keys
invoices	Charges issued to a student.	invoice_no unique; status enum
payments	Payments applied to an invoice.	invoice foreign key; cascade
notifications	Per-user inbox rows.	user foreign key; read index
books	Library catalog and copy counts.	isbn unique
library_loans	Borrowing history and due dates.	student and book foreign keys
audit_logs	Write attempts with actor and outcome.	no foreign key; history is kept
counters	Atomic sequence values.	name primary key

4.2 Relationships

- A user has at most one student profile; a student profile belongs to exactly one user.
- A student has many enrollments, attendance records, grades, invoices, and loans.
- A course has many enrollments, attendance records, and grades.
- An invoice has many payments; every payment belongs to one invoice.
- A book has many loans; every loan belongs to one student and one book.
- Audit rows keep the actor name and role even after the account is deleted.

4.3 Derived values

Attendance rates, average grades, balances, collection rates, and overdue states are computed from base rows when they are read. Attendance rate counts present and late records against all records. A grade

average uses score divided by maximum score. An invoice balance is the amount minus the sum of its payments. An invoice becomes overdue when it is unpaid and its due date has passed. A loan is overdue when it is still borrowed and its due date has passed. Nothing is denormalized, so these values cannot drift out of date.

4.4 Data lifecycle

The default development database is ephemeral: the embedded MySQL server uses a temporary data directory and the schema and seed data are recreated on every start. With DB_MODE=external and DB_AUTO_INIT=false, the same schema can be applied to a persistent server. Student deletion removes the login and cascades to the profile, enrollments, attendance, grades, invoices, payments, and loans. Audit rows are not deleted, which preserves accountability.

5. Functional Modules

5.1 Authentication and session

Login accepts an email and password, verifies an active account, compares the password with its bcrypt hash, and returns a seven-day JWT plus the user record. The client stores the token in localStorage and restores the session by calling /auth/me. A 401 clears the token and returns the user to the login screen. Deactivating a student also sets the linked account inactive, so the next sign-in is rejected.

5.2 Dashboard

The dashboard adapts to the signed-in role. Administrators see student and course totals, a seven-day attendance trend, enrollment by program, recent payments, overdue invoices, and account notifications. Librarians are routed to the library workspace. Students see their attendance rate, course count, average grade, outstanding balance, course progress, recent grades, the next invoice, and notifications.

5.3 Student records

Administrators can search by name, student number, or email, filter by status, and create or edit records. Creating a student runs one transaction that inserts the login, allocates a student number, inserts the profile, creates pending enrollments for the program and general-education courses, and issues a PHP 5,000 enrollment invoice due in 14 days. A welcome notification follows the commit.

5.4 Enrollment

Enrollment is the join between a student and a course. Each record has a status: pending, enrolled, completed, or dropped. New students start pending until their account is settled. Administrators can add, change, or remove enrollments; removing one also removes the matching attendance and grade rows in the same transaction.

5.5 Courses and grades

The course catalog stores code, title, instructor, program, units, schedule, room, and semester. Administrators manage the catalog and class rosters and post grades for enrolled students. Each grade records a term, an assessment name, a score, and a maximum score. Scores cannot be negative or exceed the maximum, and the average is always derived from the stored pairs. Students see their own courses, attendance rate, average, and individual grade items.

5.6 Attendance

Roll call is taken per course and date. The sheet lists non-dropped enrollments and prefills any existing status, so saving again updates the same row. The server accepts only real calendar dates and the four allowed states, validates each student against the roster, and upserts inside a transaction. When a student changes to absent or late, a notification is queued and written after the commit.

5.7 Library

The library module manages books, loans, and returns. A loan is allowed only when a copy is available, and the due date must be a real date that is not in the past. Recording a loan decrements availability and notifies the borrower in the same flow. Returning a loan flips its status and restores a copy. Overdue is never stored; it is derived by comparing the due date with the current date, so it cannot go stale.

5.8 Billing

Administrators create invoices with a description, amount, and due date. Students and administrators can record payments; a student may pay only their own invoice. A payment may not exceed the remaining balance. The invoice becomes partial or paid, and the balance is recalculated from the payment rows. When the student has no unpaid invoices left, all pending enrollments become enrolled and the notification reports how many were activated.

5.9 Notifications

Notifications are database rows owned by a user. The inbox supports type filters, unread filtering, mark-one-read, and mark-all-read. Administrators can announce to students, admins, or everyone. Email and SMS are simulated: the selected channels are stored and the delivery attempt is logged, but nothing leaves the machine. The sidebar polls the unread count every minute.

5.10 Reports and audit

The reports screen gathers institution-wide figures in one request: active students, attendance and collection rates, average grade, library holdings and loans, enrollment by program, attendance by course, grade distribution, six months of revenue, at-risk students, account status totals, recent enrollments and loans, and the latest audit entries. Every non-GET API request writes an audit row with actor, role, method, path, status, and IP address, including failed attempts.

5.11 Settings and appearance

Settings contains the appearance preference. Light is the default; Dark overrides the same design tokens with a deep navy palette and adjusts the few rules that cannot use tokens, such as the sidebar gradient and topbar overlay. The choice is remembered per browser and applied before first paint.

5.12 Integration flows

- Registrar to everything: one transaction creates the account, profile, pending enrollments, and invoice.
- Bursar to registrar: full settlement activates every pending enrollment.
- Attendance to student records: saved roll call updates rates and sends absence or late alerts.
- Library to portal: loans and returns change availability and notify the borrower.
- Every module to audit: write attempts are recorded for accountability.

6. API Reference

All endpoints are mounted under /api. Requests and responses use JSON. Protected endpoints expect an Authorization header with a Bearer token. Errors always use the shape {"error": "message"}. Identifiers must be positive integers and dates must be real calendar dates; invalid input returns 400 instead of a database error.

Module	Main endpoints	Access
Auth	POST /auth/login, GET /auth/me	Public login, authenticated me
Dashboard	GET /dashboard	All roles, payload varies
Students	GET/POST /students, GET/PUT/DELETE /students/:id, GET /students/:id/schedule	Admin; own profile for students
Enrollments	GET/POST /enrollments, PATCH/DELETE /enrollments/:id	Admin
Attendance	GET /attendance/me, GET/POST /attendance/sheet	Student; admin
Courses	GET/POST /courses, GET/PUT/DELETE /courses/:id, POST /courses/:id/enroll	Admin; read for students
Grades	GET /grades/me, GET/POST /grades, DELETE /grades/:id	Student; admin
Billing	GET /billing/summary, GET/POST /billing/invoices, POST /billing/invoices/:id/pay, GET /billing/payments	Admin; own invoices for students
Library	GET/POST/PUT/DELETE /library/books, GET/POST /library/loans, PATCH /library/loans/:id/return	Staff write; all read
Notifications	GET /notifications, GET /notifications/unread-count, PATCH /notifications/:id/read, POST /notifications/read-all, POST /notifications/announce	All; announce is admin
Reports	GET /reports/overview	Admin
Audit	GET /audit-logs	Admin
Health	GET /health	Public

6.1 Status codes

Code	Meaning
200	Request succeeded.
201	A record was created.
400	Validation failed: missing field, bad ID, impossible date, out-of-range value.
401	Token missing, expired, invalid, or the account is inactive.
403	The role or ownership check failed.
404	The requested record does not exist.
409	Conflict, such as a duplicate email, ISBN, course code, or an already returned loan.
500	Unexpected server error; details are logged, not returned.

6.2 Example requests

```
POST /api/auth/login
{"email": "admin@school.edu", "password": "admin123"}

POST /api/attendance/sheet
{"course_id": 6, "date": "2026-10-26",
 "records": [{"student_id": 1, "status": "late", "remarks": "Traffic"}]}
```

```
POST /api/billing/invoices/12/pay
{"amount": 1000, "method": "cash", "reference": "OR-1042"}
```

The complete endpoint reference, including every payload and role restriction, is in docs/api.md in the project repository.

7. Security and Access Control

7.1 Authentication

Passwords are hashed with bcrypt and never returned. A successful login signs a JWT with HS256 and a server-side secret; the token carries the user id, role, name, and email and expires after seven days. The client stores it in localStorage and sends it as a Bearer token. There is no refresh token and no server-side session table.

7.2 Authorization

Two layers enforce access. The React router hides or redirects pages the current role cannot use, and every API route repeats the check with adminOnly, allowRoles, or an ownership test. Student reads of profiles, schedules, attendance, grades, invoices, and loans are scoped to the student id derived from the token, never from a value supplied by the browser.

Area	Admin	Librarian	Student
Students and enrollment	Full	None	Own profile and schedule
Courses and grades	Full	None	Own records
Attendance	Roll call	None	Own records
Library	Full	Catalog and circulation	Catalog and own loans
Billing	Full	None	Own invoices and payments
Reports and audit	Full	None	None
Notifications	Own and announce	Own	Own

7.3 Input validation

Validation is centralized where possible. required rejects blank fields, parseId accepts only positive integers, and parseISODate rejects impossible dates such as February 31. Modules add their own rules: grades cannot exceed the maximum score, course units must be a whole number from 1 to 6, loan due dates cannot be in the past, payment amounts cannot exceed a balance, announcement titles are limited to 150 characters, and statuses must be in the allowed set. Invalid input returns 400 before any SQL statement runs.

7.4 Audit and transport

Every non-GET request under /api writes an audit row after the response is sent. Failed logins are included with a null actor. Audit failures are swallowed so they cannot break a request. The development server allows cross-origin requests from any origin; a production deployment should restrict CORS to the application origin and serve over HTTPS.

7.5 Known security limits

- The token is stored in localStorage, so any script injection could read it.

- A role change or deactivation does not revoke an already issued token; it applies when the token expires or the client calls /auth/me.
- There is no login rate limiting or account lockout.
- New student accounts receive a shared default password unless one is supplied.
- Payments are bookkeeping entries; there is no gateway verification or receipt signature.

8. Installation and Configuration

8.1 Prerequisites

- Node.js 20.19 or newer (22.12 or newer recommended).
- npm 10 or newer.
- Internet access the first time, to download the embedded MySQL binary.

8.2 Install and run

```
npm install        # install the root, server, and client workspaces
npm run dev        # API on :4000, Vite on :5173
# open http://localhost:5173

npm run build      # build the client into client/dist
npm start          # serve API and built client on :4000
```

8.3 Configuration

Variable	Default	Purpose
PORT	4000	API port.
JWT_SECRET	sis-dev-secret-change-me	Token signing secret. Change it before sharing.
DB_MODE	embedded	Use the embedded server or an external MySQL.
DB_AUTO_INIT	true	Apply the schema and seed on every start.
DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME	127.0.0.1, 3306, root, empty, sis_db	External MySQL connection.
MYSQL_VERSION	8.4.x	Version downloaded by the embedded server.

8.4 Seed accounts

Role	Email	Password	Landing page
Administrator	admin@school.edu	admin123	Dashboard
Librarian	librarian@school.edu	library123	Library loans
Student	maria.santos@student.edu	student123	Student dashboard

The seed dataset also creates eight students, ten courses, ten books, ten loans, attendance and grade history, invoices, payments, notifications, and audit entries so every screen has meaningful data on first launch.

9. Testing and Quality Assurance

9.1 Test suite

The project uses Node's built-in test runner, so no test framework dependency is needed. Run the suite with npm test. The API tests start the Express app on an ephemeral port with their own embedded MySQL database; they never touch a running development server or an external database.

Suite	What it covers
tests/helpers.test.mjs	ID and date validators, required fields, date helpers, money and date formatting, percentages, status and grade tones, relative time.
tests/api.test.mjs	Health, login and inactive accounts, role guards, ownership, malformed IDs, impossible dates, attendance states, grade limits, course units, student provisioning and duplicate email, invoice settlement and enrollment activation, library loans and returns, payment listing limits, announcement title limits.

9.2 Results

The current suite contains 24 tests. All 24 pass, and the production client build completes with Vite. The tests were added after a manual code review and are designed to keep the repaired input validation from regressing.

9.3 Manual acceptance walkthrough

- Sign in as admin and confirm the dashboard totals and chart render.
- Create a student, then open the profile and confirm pending enrollments and the invoice.
- Record a partial payment, then the remainder, and confirm enrollment activation.
- Save a roll call, then sign in as that student and confirm the alert and rate.
- Record and return a library loan and confirm availability changes.
- Send an announcement with email and SMS and confirm the simulated delivery log.
- Open Reports and confirm the audit entries; switch Settings to Dark and reload.
- Sign in as a student and confirm other modules redirect away.

10. Limitations and Future Work

10.1 Current limitations

- The embedded database is ephemeral and is recreated and reseeded on every start.
- Email and SMS delivery are simulated and only logged.
- Payments are recorded without a payment gateway or receipt verification.
- JWTs are stored in localStorage, last seven days, and are not revocable.
- There is no login rate limiting, refresh token, or password reset flow.
- New student accounts default to a shared password when none is supplied.
- The server and browser are assumed to share a time zone.
- There is no linter configuration; quality checks are the test suite and the build.

10.2 Suggested next steps

- Move the database to a managed MySQL instance with migrations and backups.

- Replace simulated delivery with a transactional email or SMS provider.
- Add rate limiting, refresh tokens, password reset, and role revocation.
- Add a payment gateway and immutable receipts.
- Add ESLint and Prettier, plus browser-level tests for the main flows.
- Add multi-campus scoping and configurable grading schemes.

11. Appendix: Document Control

Item	Detail
Document title	SIS System Documentation
Subject	System Integration and Architecture
Version	1.0
Date	October 2026
Academic year	2026-2027
Prepared by	SIS development team
Program	Diploma in Information Technology
Institution	College of Saint Amatiel Inc.
Source of truth	README.md, docs/architecture.md, docs/api.md, and the repository code